

24기 김가현 논문 요약

Attention Is All You Need

핵심 아이디어

기존 sequence model은 주로 RNN, LSTM, GRU를 사용. 이 구조들은 이전 시점의 hidden state를 바탕으로 다음 시점을 계산하기 때문에 순차적으로 처리해야 한다는 한계 존재. ex) 문장이 길어질수록 앞 단어의 정보를 뒤 단어까지 전달하려면 여러 step을 거쳐야 하고, 한 문장 안의 모든 위치를 동시에 계산하기도 어려웠음.

Transformer는 이 문제를 해결하기 위해 RNN과 CNN을 제거하고 Attention만으로 sequence를 처리하는 구조를 제안. 핵심은 문장 안의 각 token이 다른 모든 token을 직접 참고하도록 만드는 것.

Attention

Attention은 현재 token이 다른 token을 얼마나 참고할지 정하는 방식. 각 token으로부터 Query, Key, Value를 만듦.

- **Query(Q)**: 현재 token이 어떤 정보를 찾고 있는지 나타냄
- **Key(K)**: 각 token이 어떤 정보를 가지고 있는지 비교하기 위한 값
- **Value(V)**: 실제로 전달되는 정보

Query와 Key의 유사도를 계산해서 attention score를 만들고, 이 점수를 이용해 Value들을 가중합.

Scaled Dot-Product Attention 계산식:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

QK^T 는 Query와 Key의 유사도를 계산하는 부분. 차원 d_k 가 커질수록 dot product 값의 크기도 커질 수 있어서, $\sqrt{d_k}$ 로 나눠 값이 지나치게 커지는 것을 막음. 이후 softmax를 적용해 각 token을 얼마나 참고할지 비율로 바꾸고, 그 비율만큼 Value를 섞음. Self-Attention에서는 Q, K, V가 모두 같은 sequence에서 만들어짐. 즉 하나의 문장 안에서 각 token이 같은 문장의 다른 token을 참고함. ex) 같은 bank 라는 단어라도 주변에 money, deposit 이 있으면 금융기관 의미에 가까운 표현이 되고, river 가 있으면 강둑 의미에 가까운 표현이 될 수 있음. Attention을 거치면서 token 표현이 문맥을 반영하도록 바뀌는 것.

Multi-Head Attention

Attention을 한 번만 계산하지 않고 여러 개의 head로 나눠 병렬로 계산. 각 head는 서로 다른 projection matrix를 사용하므로 같은 문장을 보더라도 서로 다른 관계를 학습할 수 있음. 어떤 head는 주어-동사 관계를 보고, 다른 head는 멀리 떨어진 단어 간 의미 관계를 보는 식.

각 head 식:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

여러 head의 결과를 이어 붙인 뒤 다시 선형변환.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

논문의 base Transformer는 8개의 attention head를 사용.

Transformer 구조

Transformer는 기본적으로 Encoder-Decoder 구조를 사용. Encoder는 입력 sequence를 이해해서 표현을 만들고, Decoder는 그 표현을 이용해 출력 sequence를 한 token씩 생성.

Encoder

Encoder는 동일한 layer를 6개 쌓은 구조. 각 layer는 크게 두 부분으로 구성됨.

- Multi-Head Self-Attention
- Position-wise Feed-Forward Network

각 sub-layer 뒤에는 residual connection과 layer normalization을 적용.

$$\text{LayerNorm}(x + \text{Sublayer}(x))$$

즉 sub-layer의 출력만 사용하는 것이 아니라 입력 x 도 그대로 더해줌. ResNet의 residual connection과 같은 아이디어. Feed-Forward Network는 각 위치에 독립적으로 동일하게 적용.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

base model에서는 입력과 출력 차원이 512이고, 중간 hidden dimension은 2048.

Decoder

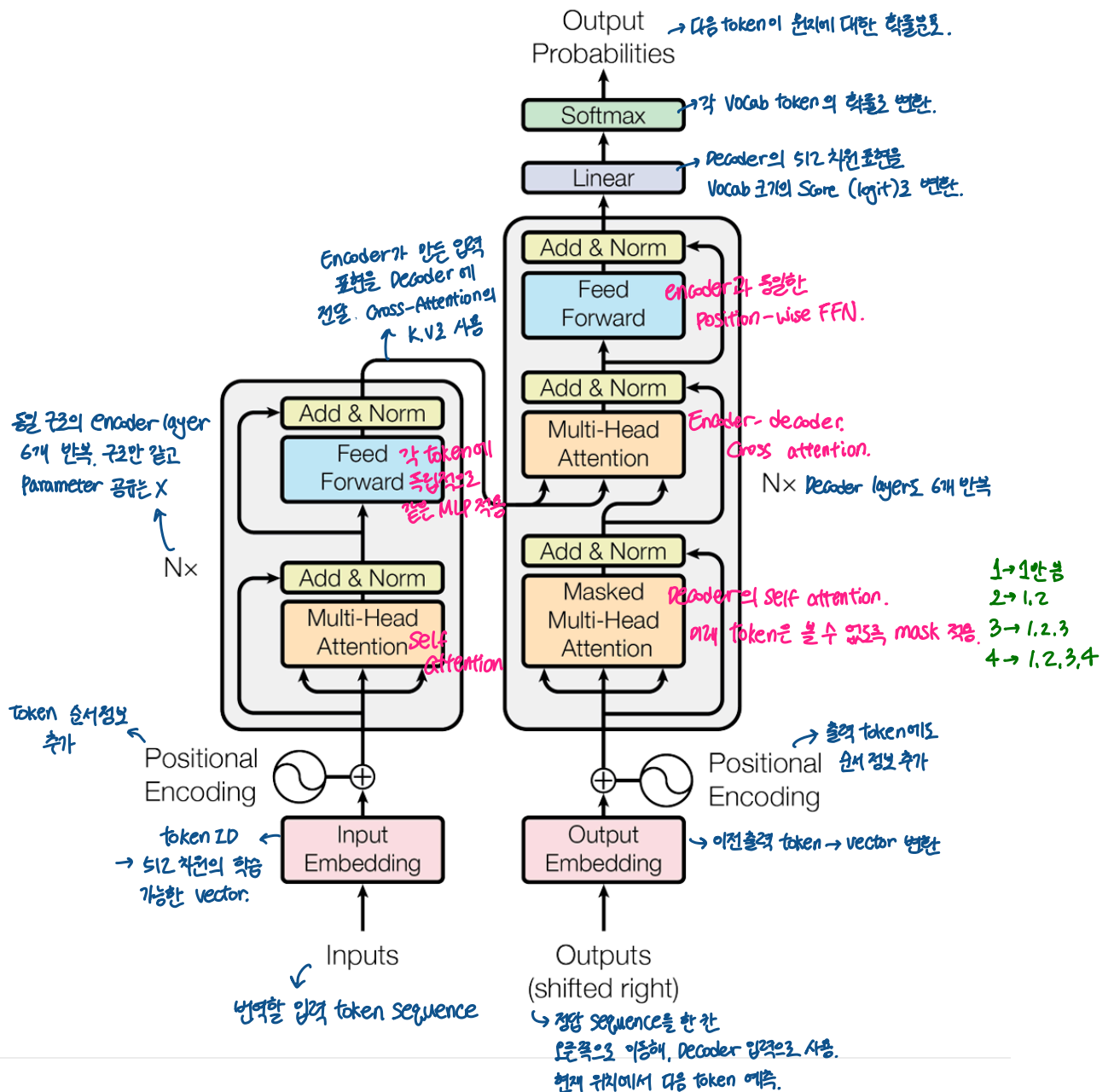
Decoder도 동일한 layer를 6개 쌓음. Encoder와 달리 하나의 layer 안에 세 부분 존재.

- Masked Multi-Head Self-Attention
- Encoder-Decoder Attention
- Feed-Forward Network

Decoder는 다음 token을 생성할 때 미래 token을 미리 보면 안 되므로 self-attention에 mask를 사용. ex) 세 번째 token을 예측할 때는 첫 번째와 두 번째 token까지만 참고할 수 있고, 네 번째 이후 위치는 보지 못하게 함. Attention score에서 볼 수 없는 위치를 $-\infty$ 로 만들어 softmax 이후 weight가 0이 되도록 처리. Encoder-Decoder Attention에서는 Query가 Decoder에서 오고, Key와 Value는 Encoder 출력에서 옴. 즉 Decoder가 현재 출력 token을 생성할 때 입력 문장의 어느 부분을 참고해야 하는지 결정하는 역할.

정리하면 Transformer 안에서는 다음 세 종류의 attention이 사용됨.

- Encoder Self-Attention: 입력 문장 내부 관계를 봄
- Decoder Masked Self-Attention: 지금까지 생성한 출력끼리 관계를 봄
- Encoder-Decoder Attention: Decoder가 Encoder의 입력 표현을 참고함



Positional Encoding

Transformer는 RNN처럼 token을 순서대로 처리하지 않기 때문에, Attention만으로는 token의 순서를 알 수 없음. 그래서 token embedding에 위치 정보를 나타내는 Positional Encoding을 더함.

$$\text{Input Representation} = \text{Token Embedding} + \text{Positional Encoding}$$

논문에서는 sine과 cosine 함수를 사용.

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

$$PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$

각 위치마다 서로 다른 패턴의 벡터가 만들어지기 때문에 모델이 token의 위치와 상대적인 순서를 구분 가능. 논문에서는 학습 가능한 positional embedding도 비교했는데 성능은 거의 비슷. 최종적으로는 학습 때보다 긴 sequence에도 적용될 가능성을 고려해 sinusoidal positional encoding을 사용.

왜 Self-Attention?

논문에서는 Self-Attention, RNN, CNN을 계산량, 병렬화, 먼 위치 간 정보 전달 관점에서 비교. RNN은 한 위치의 계산이 끝나야 다음 위치를 계산할 수 있으므로 최소 $O(n)$ 개의 순차 연산이 필요. 반면 Self-Attention은 모든 위치를 한 번에 계산할 수 있어서 순차 연산 수가 $O(1)$. 또한 RNN에서 멀리 떨어진 두 token이 정보를 주고받으려면 여러 hidden state를 거쳐야 하지만 Self-Attention에서는 두 token이 직접 연결될 수 있음. 따라서 long-range dependency를 학습하기에 유리함.

Self-Attention의 계산량:

$$O(n^2d)$$

이므로 sequence 길이 n 이 매우 길어지면 계산량이 빠르게 증가함.

논문에서는 당시 기계번역에서 일반적인 조건인 $n < d$ 에서는 Self-Attention이 RNN보다 효율적이라고 설명.

학습 및 결과

영어-독일어 번역에는 약 450만 문장 쌍의 WMT 2014 데이터셋을 사용했고, 영어-프랑스어 번역에는 약 3600만 문장 쌍을 사용. Optimizer는 Adam을 사용했으며 초반 4000 step 동안 learning rate를 점차 증가시키는 warmup을 적용. 이후에는 step 수의 inverse square root에 비례해 learning rate를 감소시킴. Regularization으로 dropout과 label smoothing을 사용. Transformer big 모델은 WMT 2014 영어-독일어 번역에서 BLEU 28.4를 기록해 당시 기존 최고 성능을 2 BLEU 이상 개선. 영어-프랑스어 번역에서도 높은 성능을 보였고, 기존 RNN이나 CNN 기반 모델보다 훨씬 적은 training cost로 좋은 결과.

구성 요소를 바꿔본 실험에서는 single-head보다 multi-head attention이 더 좋은 성능을 보였고, head 수를 지나치게 늘리는 것도 성능 향상으로 이어지지는 않았음. 모델 크기를 키우면 전반적으로 성능이 좋아졌고, dropout은 overfitting을 줄이는 데 도움이 됐음. sinusoidal positional encoding과 learned positional embedding은 거의 비슷한 성능을 보임. 번역 외에 English constituency parsing에도 적용했는데 별도의 큰 구조 변경 없이 좋은 성능을 보여 Transformer가 번역에만 한정된 구조가 아니라는 점도 확인.

핵심: sequence를 반드시 RNN처럼 순차적으로 처리할 필요는 없으며, Self-Attention을 이용하면 각 위치의 관계를 직접 계산하면서 병렬 처리도 가능하다.